
Bài 3

Tối ưu hóa Cơ sở dữ liệu

Nội dung

- 
- 1. Giới thiệu về database tuning**
 - 2. Tối ưu hóa truy vấn dữ liệu (query tuning)**
 - 3. Sử dụng INDEX**
 - 4. Giới thiệu Stored procedure và Trigger**

Giới thiệu về database tuning

- Tinh chỉnh cơ sở dữ liệu (**database tuning**)
 - Hoạt động giúp ứng dụng cơ sở dữ liệu chạy nhanh hơn
 - Thao tác I/O nhanh hơn: INSERT, SELECT, DELETE, UPDATE,
 - Và cũng như tối ưu hóa không gian lưu trữ, sử dụng mạng, v.v.
- Các thông số tinh chỉnh (tuning parameters)
 - Tất cả các khía cạnh giúp tăng hiệu quả cho cơ sở dữ liệu
 - Sử dụng đĩa lưu trữ nhanh hơn (Faster disk)
 - Cung cấp nhiều bộ nhớ hơn (More RAM)
 - Sử dụng chỉ mục hiệu quả (Effective index)
 - Viết các truy vấn dữ liệu tốt (Good queries)
 - ...
 - Luôn xem xét chi phí/sự đánh đổi
 - Một số biện pháp tốn chi phí thời gian thấp và mang lại lợi ích cao

Giới thiệu về database tuning

- Năm nguyên tắc tinh chỉnh cơ bản
 - Suy nghĩ toàn cục, khắc phục cục bộ
 - Phân vùng để phá vỡ nút cổ chai
 - Chi phí khởi động cao, chi phí vận hành thấp
 - Công việc phía sever hãy xử lý tại server
 - Sẵn sàng cho sự đánh đổi

Tối ưu hóa lược đồ và các kiểu dữ liệu

- Lựa chọn kiểu dữ liệu tối ưu
 - Nhỏ hơn thường tốt hơn (smaller is usually better)
 - Nhanh hơn, ít dung lượng hơn trên đĩa, trong bộ nhớ và trong cache của CPU
 - Các kiểu đơn giản thường tốt hơn (simple is good)
 - Ví dụ: chi phí thao tác với số nguyên rẻ hơn so với ký tự
 - Tránh các giá trị NULL nếu có thể
 - Các cột có thể chứa giá trị null khiến cho việc tạo chỉ mục, thống kê chỉ mục và so sánh giá trị phức tạp hơn

Tối ưu hóa lược đồ và các kiểu dữ liệu

■ Lựa chọn kiểu dữ liệu tối ưu

- Các kiểu số:

- Whole Numbers

- * TINYINT: 8 bits
- * SMALLINT: 16 bits
- * MEDIUMINT: 24 bits
- * INT: 32 bits
- * BIGINT: 64 bits
- * tùy chọn có thuộc tính UNSIGNED

- Real Numbers

- * DECIMAL: lưu trữ các số chính xác. Ví dụ. DECIMAL(18, 9) chỉ khi cần kết quả chính xác cho các số phân số
- * FLOAT: 32 bits
- * DOUBLE: 64 bits

Tối ưu hóa lược đồ và các kiểu dữ liệu

■ Lựa chọn kiểu dữ liệu tối ưu

- Các kiểu chuỗi ký tự:

- VARCHAR

- * Chuỗi ký tự có độ dài thay đổi
- * Kiểu dữ liệu chuỗi phổ biến nhất
- * Cần ít dung lượng lưu trữ hơn các loại có độ dài cố định (ngoại trừ công cụ lưu trữ MyISAM)
- * Sử dụng thêm 1 hoặc 2 byte để ghi lại độ dài của giá trị
- * Cần làm thêm khi dữ liệu tăng lên
- * VARCHAR(n): giữ n càng nhỏ càng tốt (MySQL thường phân bổ các khối bộ nhớ có kích thước cố định để giữ các giá trị bên trong)

- CHAR

- * Chiều dài cố định
- * Hữu ích để lưu trữ các chuỗi rất ngắn có độ dài gần bằng nhau

- **Kinh nghiệm**

- * Sử dụng ENUM thay vì kiểu chuỗi nếu có thể
- * Với dữ liệu là địa chỉ IP: ở dạng số nguyên không dấu, không phải chuỗi

Tối ưu hóa lược đồ và các kiểu dữ liệu

■ Lựa chọn kiểu dữ liệu tối ưu

- Các kiểu có kích thước lớn: BLOB and TEXT
 - Các kiểu này dùng cho lượng lớn dữ liệu dưới dạng chuỗi nhị phân hoặc chuỗi ký tự
 - * TINYTEXT, SMALL TEXT, TEXT, MEDIUMTEXT và LONGTEXT
 - * TINYBLOB, SMALLBLOB, BLOB, MEDIUMBLOB và LONGBLOB
 - MySQL xử lý từng giá trị BLOB và TEXT như một đối tượng
 - InnoDB có thể sử dụng vùng lưu trữ “bên ngoài” riêng biệt
 - Memory storage engine không hỗ trợ các loại BLOB và TEXT
- **Kinh nghiệm:** Tránh sử dụng các loại BLOB và TEXT trừ khi cần thiết

Tối ưu hóa lược đồ và các kiểu dữ liệu

- Lựa chọn kiểu dữ liệu tối ưu
 - Các kiểu Date and Time
 - DATETIME
 - * lưu trữ ngày và giờ được đóng gói thành một số nguyên trong
 - * Định dạng YYYYMMDDHHMMSS
 - TIMESTAMP
 - * số giây đã trôi qua kể từ nửa đêm, ngày 1 tháng 1 năm 1970 (GMT)
 - * tiết kiệm không gian hơn DATETIME

Tối ưu hóa lược đồ và các kiểu dữ liệu

■ Lựa chọn kiểu dữ liệu tối ưu

- Các kiểu cho định danh - identifier (primary key)

- Nên xem xét không chỉ loại lưu trữ mà còn cả cách MySQL thực hiện tính toán và so sánh
- Số nguyên
 - * thường là lựa chọn tốt nhất cho các trường định danh
 - * nhanh chóng và hỗ trợ AUTO_INCREMENT
- ENUM và SET
 - * Lựa chọn kém hơn cho các cột định danh
- Các loại chuỗi
 - * Tránh sử dụng nếu có thể
 - * Tránh các chuỗi "ngẫu nhiên", chẳng hạn như các chuỗi được tạo bởi MD5(), SHA1() hoặc UUID(). Chúng làm chậm các truy vấn INSERT, SELECT, khiến bộ đệm hoạt động kém

Tối ưu hóa lược đồ và các kiểu dữ liệu

■ Tối ưu hóa lược đồ dữ liệu

- Để các truy vấn dữ liệu thực hiện nhanh chóng thì dữ liệu cần đảm bảo luôn sẵn có. Một phương pháp thiết kế đơn giản nhất là sử dụng một bảng lớn và lưu trữ tất cả dữ liệu
- Nhưng vấn đề với điều này là gì?
 - **Bất thường** khi thực hiện các thao tác: thêm, sửa, xóa dữ liệu
 - **Dư thừa** dữ liệu, tốn không gian lưu trữ
- **Chuẩn hóa** là quá trình loại bỏ các điểm bất thường và dư thừa khỏi CSDL. Mỗi dạng chuẩn được thiết kế để loại bỏ một hoặc một số điểm bất thường
- Ngược lại với chuẩn hóa là **phi chuẩn hóa - Denormalization**
 - Join số lượng bảng lớn hơn cần thêm thao tác vào/ra đĩa (I/O) và logic xử lý
 - Giảm tốc độ hệ thống
 - Xung đột giữa hiệu quả thiết kế, yêu cầu thông tin và tốc độ xử lý thường được giải quyết thông qua các thỏa hiệp có thể bao gồm việc phi chuẩn hóa - Denormalization
 - → Có thể cần phải chấp nhận mất đi một số lợi ích của thiết kế được chuẩn hóa hoàn toàn để ưu tiên cho hiệu suất

Tối ưu hóa lược đồ và các kiểu dữ liệu

- Tối ưu hóa lược đồ dữ liệu
 - Các bước denormalization
 - Kết hợp các mối quan hệ 1-1 (1:1)
 - Sao chép các thuộc tính không khóa trong mỗi quan hệ một-nhiều (1:*) để giảm liên kết
 - Sao chép thuộc tính khóa ngoại trong mỗi quan hệ một-nhiều (1:*) để giảm kết nối
 - Sao chép thuộc tính trong mỗi quan hệ nhiều-nhiều (*:*) để giảm phép nối
 - Giới thiệu các nhóm lặp lại
 - Tạo bảng trích xuất
 - Phân vùng quan hệ

Thực hành tối ưu hóa lược đồ và các kiểu dữ liệu

- Lược đồ CSDL dưới đây đang ở dạng chuẩn nào?
- Xác định các kiểu dữ liệu phù hợp cho các trường trong quan hệ?
- Phi chuẩn hóa quan hệ cho một số truy vấn dữ liệu.

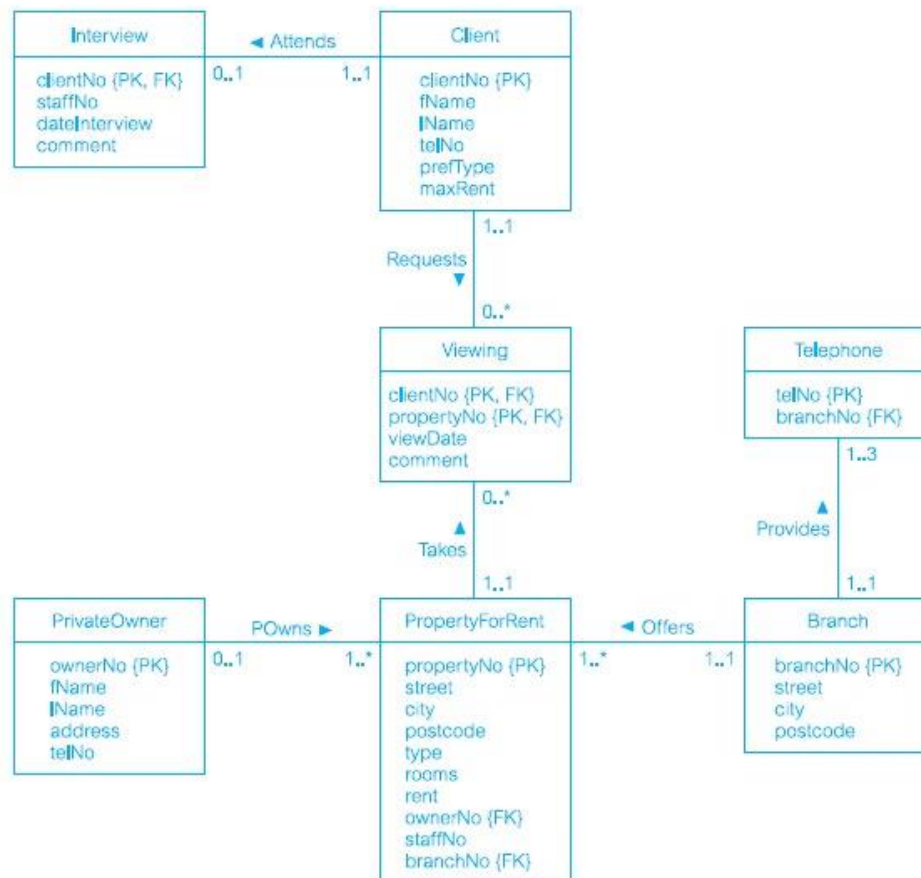


Figure 18.1
(a) Sample relation diagram.]

Nội dung

1. Giới thiệu về database tuning



2. Tối ưu hóa truy vấn dữ liệu (query tuning)

3. Sử dụng INDEX

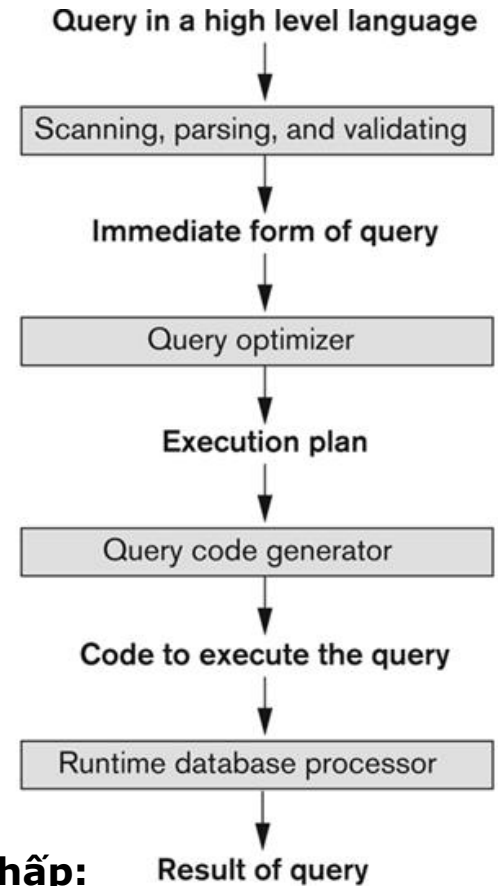
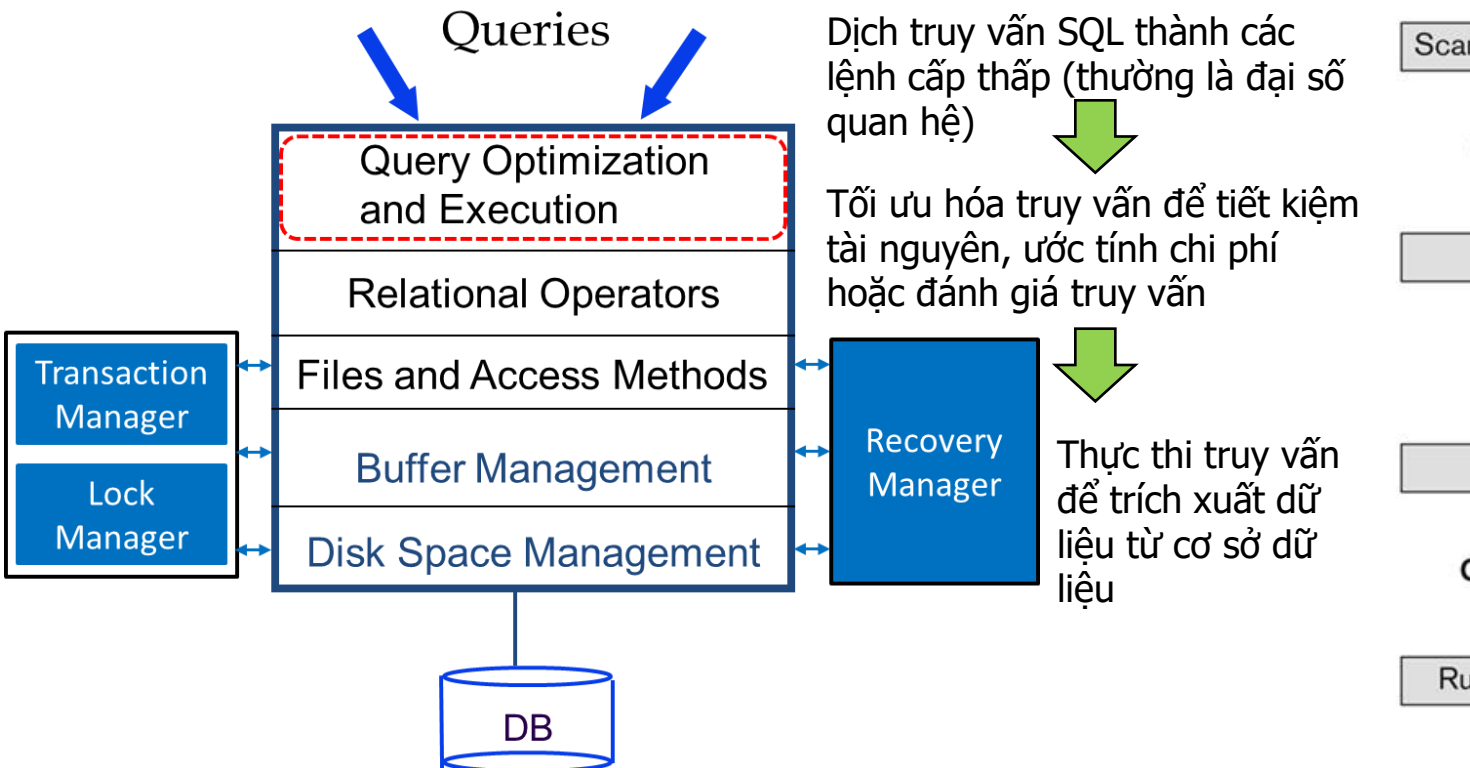
4. Giới thiệu Stored procedure và Trigger

Tối ưu hóa truy vấn dữ liệu

- **Tối ưu truy vấn**: là quá trình biến đổi truy vấn sang một truy vấn tương đương (nghĩa là cho cùng một kết quả) để giảm thiểu thời gian tính toán.
- Việc tối ưu không nhất thiết phải trên mọi khả năng có thể của các cách cài đặt của các câu hỏi.
 - Chúng ta có thể thỏa hiệp giữa giải thuật phức tạp hoặc/và tốn kém không gian lưu trữ với việc tiết kiệm thời gian xử lý.
- Viết lại truy vấn để chạy nhanh hơn là điều đầu tiên cần làm nếu truy vấn chậm
- Các phương pháp điều chỉnh khác
 - Thêm chỉ mục - index
 - Thay đổi lược đồ (chuẩn hóa về 3, 4 NF, v.v.)
 - Thay đổi độ dài giao dịch

Quá trình xử lý truy vấn

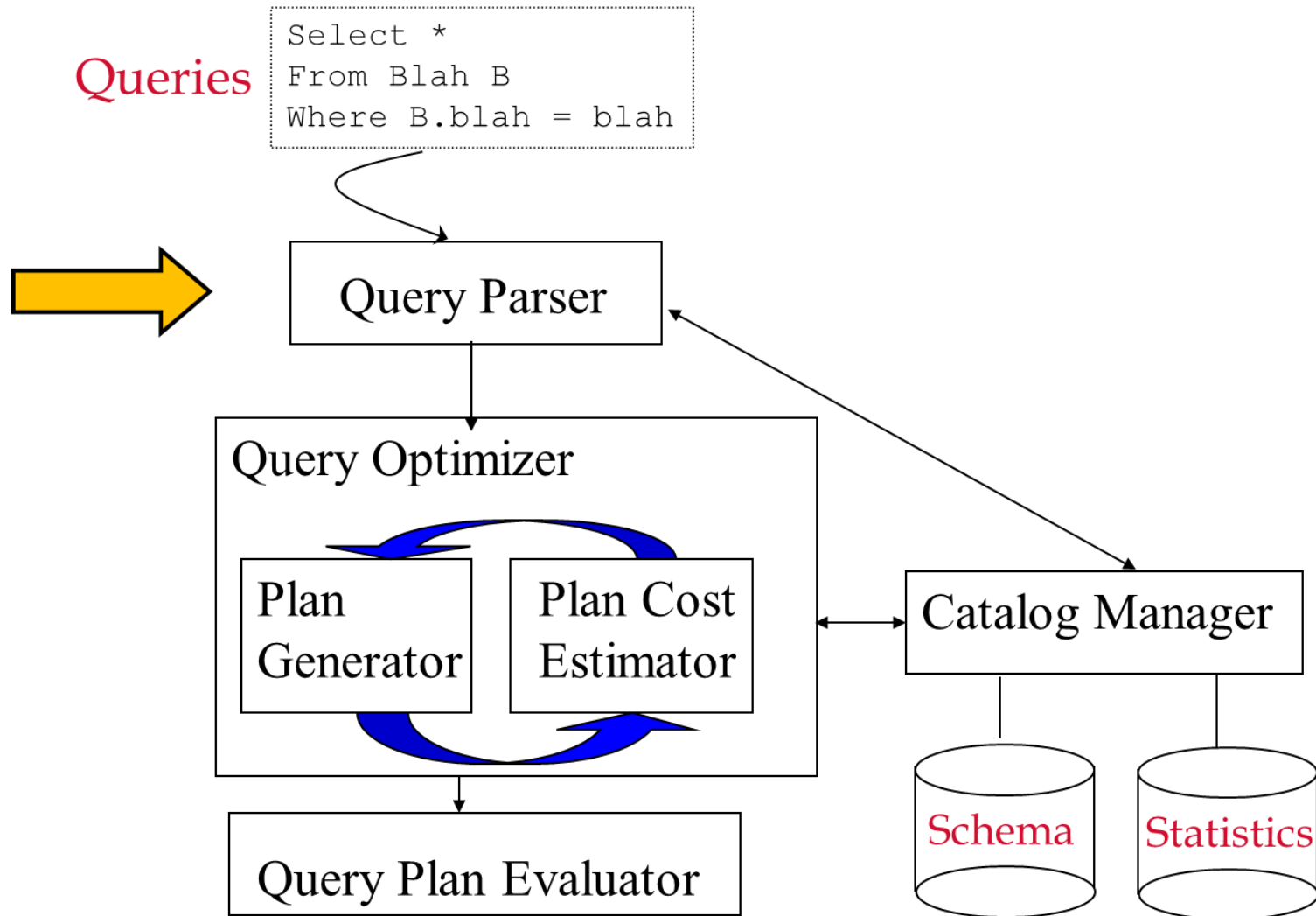
Quá trình hoặc các hoạt động liên quan đến việc truy xuất dữ liệu từ cơ sở dữ liệu



Tối ưu hoá truy vấn dựa trên chi phí thấp:

- Ý tưởng chính:
 - Giảm chi phí cho các thao tác
 - Giảm chi phí cho các phương án thực thi
 - Lựa chọn phương án có chi phí thấp nhất

Quá trình xử lý truy vấn



Quá trình xử lý truy vấn

- Tại sao chúng ta cần tinh chỉnh truy vấn (query tuning)?
Tại sao trình tối ưu hóa truy vấn (query optimizer) là không đủ?
- Trình tối ưu hóa không hoàn hảo:
 - phép biến đổi chỉ tạo ra một tập hợp con của tất cả các kế hoạch truy vấn có thể
 - chỉ một tập hợp con của các chú thích có thể được xem xét
 - chi phí của kế hoạch truy vấn chỉ có thể được ước tính
- Tinh chỉnh truy vấn: Giúp trình tối ưu hóa truy vấn hoạt động dễ dàng hơn!

Quá trình xử lý truy vấn

- Tại sao query lại chậm? Bạn đã thực sự lấy đúng những dữ liệu mình cần chưa?
- Các yếu tố ảnh hưởng tới tốc độ câu truy vấn:
 - Không hoặc thiếu sử dụng các lợi ích của Indexes.
 - Các câu truy vấn được viết không tốt.
 - Trả về các dữ liệu không cần thiết.
 - Không/thiếu tận dụng được I/O striping.
 - Các thống kê lỗi thời hoặc thiếu các thống kê hữu ích.
 - Thiếu bộ nhớ vật lý.
 - Kết nối mạng chậm.
 - Các câu truy vấn chuyển số lượng dữ liệu lớn từ server đến client.

Lệnh EXPLAIN

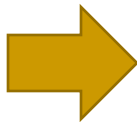
- Chỉ ra các truy vấn có vấn đề? Những câu truy vấn nào nên được viết lại?
 - Viết lại các truy vấn chạy “quá chậm”
- Làm thế nào để tìm những truy vấn này?
 - Vấn đề truy vấn truy cập đĩa quá nhiều, ví dụ: truy vấn điểm quét toàn bộ bảng
 - Có thể xem xét kế hoạch truy vấn và thấy rằng các chỉ mục liên quan không được sử dụng

Lệnh EXPLAIN

- Explain là câu lệnh được sử dụng để thu được kế hoạch thực thi truy vấn, hay MySQL sẽ thực thi truy vấn như thế nào.
- Cú pháp: thêm EXPLAIN vào trước câu truy vấn SELECT

```
explain select * from t1, t2 where ...
union
select * from t10, t11
where t10.col in (select t20.col from t20 where ...);
```

id	select_type	table	type
1	PRIMARY	t1	ALL
1	PRIMARY	t2	ALL
2	UNION	t10	ALL
2	UNION	t11	ALL
3	DEPENDENT SUBQUERY	t20	ALL
NULL	UNION RESULT	<union1,2>	ALL



EXPLAIN output column

Column	Meaning
<u>id</u>	The SELECT identifier
<u>select_type</u>	The SELECT type
<u>table</u>	The table for the output row
<u>type</u>	The join type
<u>possible_keys</u>	The possible indexes to choose
<u>key</u>	The index actually chosen
<u>key_len</u>	The length of the chosen key
<u>ref</u>	The columns compared to the index
<u>rows</u>	Estimate of rows to be examined
<u>Extra</u>	Additional information

Lệnh EXPLAIN

■ Lệnh EXPLAIN cung cấp các thông tin sau

- MySQL sẽ đưa ra cách thức mà nó thực hiện câu lệnh SELECT (các bảng được **JOIN như thế nào? Theo thứ tự nào?** – nhớ rằng MySQL sẽ **tự động tìm ra thứ tự JOIN** các bảng sao cho thích hợp nhất)
- Bằng cách quan sát các thông tin trên, bạn sẽ biết phải **thêm các Index** nào vào bảng để tăng tốc độ Query
- Đối với câu lệnh có JOIN, các bảng có liên quan sẽ được liệt kê **theo thứ tự mà MySQL sẽ đọc** khi thực hiện lệnh SELECT
- **select_type**
 - **SIMPLE**: không sử dụng UNION
 - **PRIMARY**: select đầu tiên trong câu lệnh UNION
 - **UNION**: các select tiếp theo
 - **DEPENDENT UNION**: các select tiếp theo trong câu lệnh UNION phụ thuộc vào select ngoài
 - **SUBQUERY**: select đầu tiên trong subquery
 - **DEPENDENT SUBQUERY**: select đầu tiên phụ thuộc vào câu lệnh select ngoài
 - **DERIVED**: câu lệnh select đặt ở mệnh đề FROM

Lệnh EXPLAIN

■ Lệnh EXPLAIN cung cấp các thông tin sau type

- system: bảng chỉ có 1 dòng
- const: bảng có nhiều nhất 1 dòng phù hợp, được đọc đầu tiên. Được sử dụng khi so sánh tất cả các PRIMARY/UNIQUE với một hằng số

```
SELECT * FROM const_table WHERE primary_key=1;
```

```
SELECT * FROM const_table
```

```
WHERE primary_key_part1=1 AND primary_key_part2=2;
```

- eq-ref: từng dòng của bảng này sẽ được đọc để kết hợp với bảng đứng trước. Được sử dụng khi tất cả các index có mặt trong phép so sánh với toán tử =

```
SELECT * FROM ref_table,other_table
```

```
WHERE ref_table.key_column=other_table.column;
```

```
SELECT * FROM ref_table,other_table
```

```
WHERE ref_table.key_column_part1=other_table.column
```

```
AND ref_table.key_column_part2=1;
```

Lệnh EXPLAIN

- Lệnh EXPLAIN cung cấp các thông tin sau

type:

- ref: Tất cả các dòng phù hợp của bảng được đọc để kết hợp với dòng hiện tại của bảng trước. Được sử dụng khi biểu thức so sánh có ít nhất 1 index của bảng ref (kết quả cho ra hơn 1 dòng phù hợp)

```
SELECT * FROM ref_table WHERE key_column=expr;
```

```
SELECT * FROM ref_table,other_table
```

```
WHERE ref_table.key_column=other_table.column;
```

```
SELECT * FROM ref_table,other_table
```

```
WHERE ref_table.key_column_part1=other_table.column
```

```
AND ref_table.key_column_part2=1;
```

- ref_or_null: Giống ref nhưng mở rộng thêm điều kiện tìm kiếm IS NULL

Lệnh EXPLAIN

■ Lệnh EXPLAIN cung cấp các thông tin sau

type:

- range: Các dòng của bảng được chọn trong 1 khoảng của index
`SELECT * FROM range_table WHERE key_column = 10;`
`SELECT * FROM range_table WHERE key_column BETWEEN 10 and 20;`
`SELECT * FROM range_table WHERE key_column IN (10,20,30);`
`SELECT * FROM range_table WHERE key_part1= 10 and key_part2 IN (10,20,30);`
- index: Giống ALL, ngoài việc cây index được sử dụng nên tốc độ sẽ nhanh hơn. Được sử dụng khi chỉ liệt kê trường index của bảng.
- ALL: Cả bảng sẽ được sử dụng để kết hợp với bảng đứng trước. Trừ phi là bảng đầu tiên, trong tất cả các trường hợp còn lại đều không tốt (nên tránh bằng cách sử dụng các trường index)

Lệnh EXPLAIN

■ Lệnh EXPLAIN cung cấp các thông tin sau

possible key

- Chỉ ra các index có thể được sử dụng. Tuy nhiên, nếu thứ tự các bảng bị thay đổi trong quá trình tối ưu hóa thì có thể index này sẽ không có ích nữa.

key

- Chỉ ra các index thực sự được sử dụng

key length

- Độ dài của khóa được sử dụng

ref

- Cột hoặc hằng được sử dụng để chọn các dòng trong bảng

rows

- Số dòng MySQL phải kiểm tra để thực hiện câu lệnh

Lệnh EXPLAIN

■ Lệnh EXPLAIN cung cấp các thông tin sau extra

- Distinct: Chỉ tìm 1 kết quả thích hợp
- Not exists: Sử dụng trong phép kết nối LEFT JOIN, khi chọn các dòng của bảng bên trái không phù hợp với tất cả các dòng của bảng bên phải
- Using file sort: Phải thực hiện thêm 1 bước: duyệt qua cả bảng, lưu lại khóa sắp xếp và con trỏ đến dòng tương ứng. Sau đó mới tiến hành JOIN như bình thường
- Using where: Mệnh đề WHERE được sử dụng để giới hạn số lượng các dòng. Nếu bảng của bạn có type=ALL và ko sử dụng mệnh đề WHERE cũng như index thì tất cả các dòng sẽ được liệt kê ra

Chiến lược tối ưu truy vấn

■ Tránh DISTINCT

- Sử dụng DISTINCT yêu cầu sắp xếp hoặc chi phí khác.
- Nếu không cần thiết thì nên tránh
- Ví dụ: Tìm nhân viên làm việc trong bộ phận hệ thống thông tin.
SELECT DISTINCT MaNhanVien
FROM NhanVien
WHERE MaPhong = 1;
 - Trong câu truy vấn này DISTINCT không cần thiết: MaNhanVien là một khóa của Nhân viên nên nó không trùng lặp giá trị

Chiến lược tối ưu truy vấn

- Đơn giản hóa các điều kiện với các biến đổi tương đương
 - Biến đổi các điều kiện thành các điều kiện tương đương nhưng đơn giản hơn
 - * Ý tưởng chính: **col1=const AND col1=col2** → **col2=const**
 - Bỏ các ngoặc không cần thiết
 - $((a \text{ AND } b) \text{ AND } c \text{ OR } (((a \text{ AND } b) \text{ AND } (c \text{ AND } d))))$
 - > $(a \text{ AND } b \text{ AND } c) \text{ OR } (a \text{ AND } b \text{ AND } c \text{ AND } d)$
 - Chuyển các phép so sánh các cột thành hằng (nếu được)
 - $(a < b \text{ AND } b = c) \text{ AND } a = 5 \rightarrow b > 5 \text{ AND } b = c \text{ AND } a = 5$
 - Loại bỏ các điều kiện dư thừa
 - $(B \geq 5 \text{ AND } B = 5) \text{ OR } (B = 6 \text{ AND } 5 = 5) \text{ OR } (B = 7 \text{ AND } 5 = 6) \rightarrow B = 5 \text{ OR } B = 6$

```
Ví dụ: select * from t1
where t1.col1=4 and t1.col1=t1.col2 and
t1.col3 like concat(t1.col2, ' %');
```

Chiến lược tối ưu truy vấn

- Chuyển đổi các truy vấn con (subquery)
 - Nhiều hệ thống xử lý truy vấn con không hiệu quả. Các subquery không tương quan: thuộc tính của truy vấn bên ngoài không được sử dụng trong truy vấn bên trong.
 - IN→EXISTS rewrite
 - * `x IN (SELECT y ... ) y`
 - * `EXISTS (SELECT 1 FROM .. WHERE|HAVING x=y)`
 - MIN/MAX rewrite
 - * `x > ANY (SELECT) → x > (SELECT max(...) ...)`
 - **Ví dụ:** SELECT MaNhanVien
 - FROM NhanVien
 - WHERE MaPhong IN (SELECT MaPhong FROM PhongBan)
 - **Viết lại:** SELECT MaNhanVien
 - FROM NhanVien n, PhongBan p
 - WHERE n.MaPhong = p.MaPhong

Chiến lược tối ưu truy vấn

■ Sử dụng Temporary tables

- Các bảng tạm thời có thể gây hại theo những cách sau:
 - buộc các thao tác được thực hiện theo thứ tự dưới mức tối ưu (trình tối ưu hóa thường hoạt động rất tốt!)
 - tạo các bảng tạm thời gây ra cập nhật chỉ mục – nút cổ chai kiểm soát tương tranh có thể xảy ra
 - hệ thống có thể bỏ lỡ cơ hội sử dụng chỉ mục
- Các tình huống sử dụng bảng tạm tốt:
 - để viết lại các truy vấn con tương quan phức tạp
 - để tránh ORDER BY và quét trong các trường hợp cụ thể

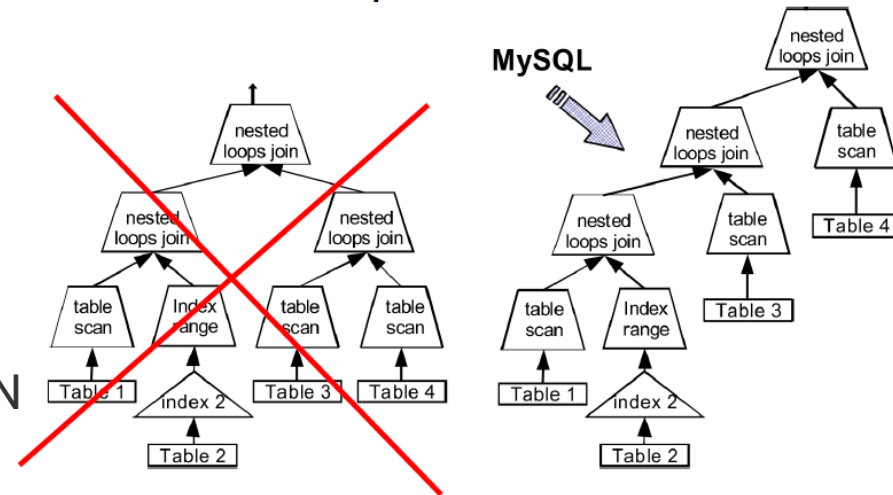
Chiến lược tối ưu truy vấn

- Sử dụng chỉ mục khi kết nối (join) các bảng
 - MySQL thực hiện mệnh đề JOIN theo thuật toán “nested loop join”
 - MySQL sẽ tự động tìm ra thứ tự JOIN các bảng sao cho thích hợp nhất

Hãy cố gắng JOIN ít bảng hơn trong truy vấn. Một câu lệnh SQL với quá nhiều JOIN có thể không hoạt động tốt.

```
select * from t1, t2
where (t1.col1='foo' and t2.col2=1 and t2.col3=t1.col3;
```

```
for each record R1 in t1 {
  for each record R2 in t2 {
    if (R1.col1='foo' && R2.col2=1 && R1.col3=R2.col3) {
      pass (R1,R2) to output;
    }
  }
}
```



Chiến lược tối ưu truy vấn

- Index các cột dùng trong lệnh 'WHERE', 'ORDER BY' và 'GROUP BY'
 - Nhận dạng duy nhất các bản ghi, giúp máy chủ MySQL lấy bản ghi từ CSDL nhanh hơn và cũng rất có ích khi sắp xếp các bản ghi
- Tối ưu hóa câu lệnh bằng Union
 - Các câu truy vấn so sánh với 'like', 'or'. Khi sử dụng 'or' quá nhiều có thể mysql sẽ phải search toàn bộ bảng để tìm kiếm bản ghi. Union có thể giúp câu truy vấn trở nên nhanh hơn đặc biệt là trong trường hợp đã đánh index một cách hợp lý.
- Tránh sử dụng câu truy vấn dùng like với '%' phía trước
 - Ví dụ: `select * from students where first_name like '%A' ;`
 - index có thể không có tác dụng trong trường hợp này, truy vấn vẫn cần phải duyệt toàn bộ bảng để tìm kiếm các bản ghi thỏa mãn yêu cầu. Trong trường hợp này hãy xem xét kỹ nếu có thật sự cần dùng % ở phía trước nếu không hãy loại bỏ nó hoặc xem xét có thể dùng full-text index

Chiến lược tối ưu truy vấn

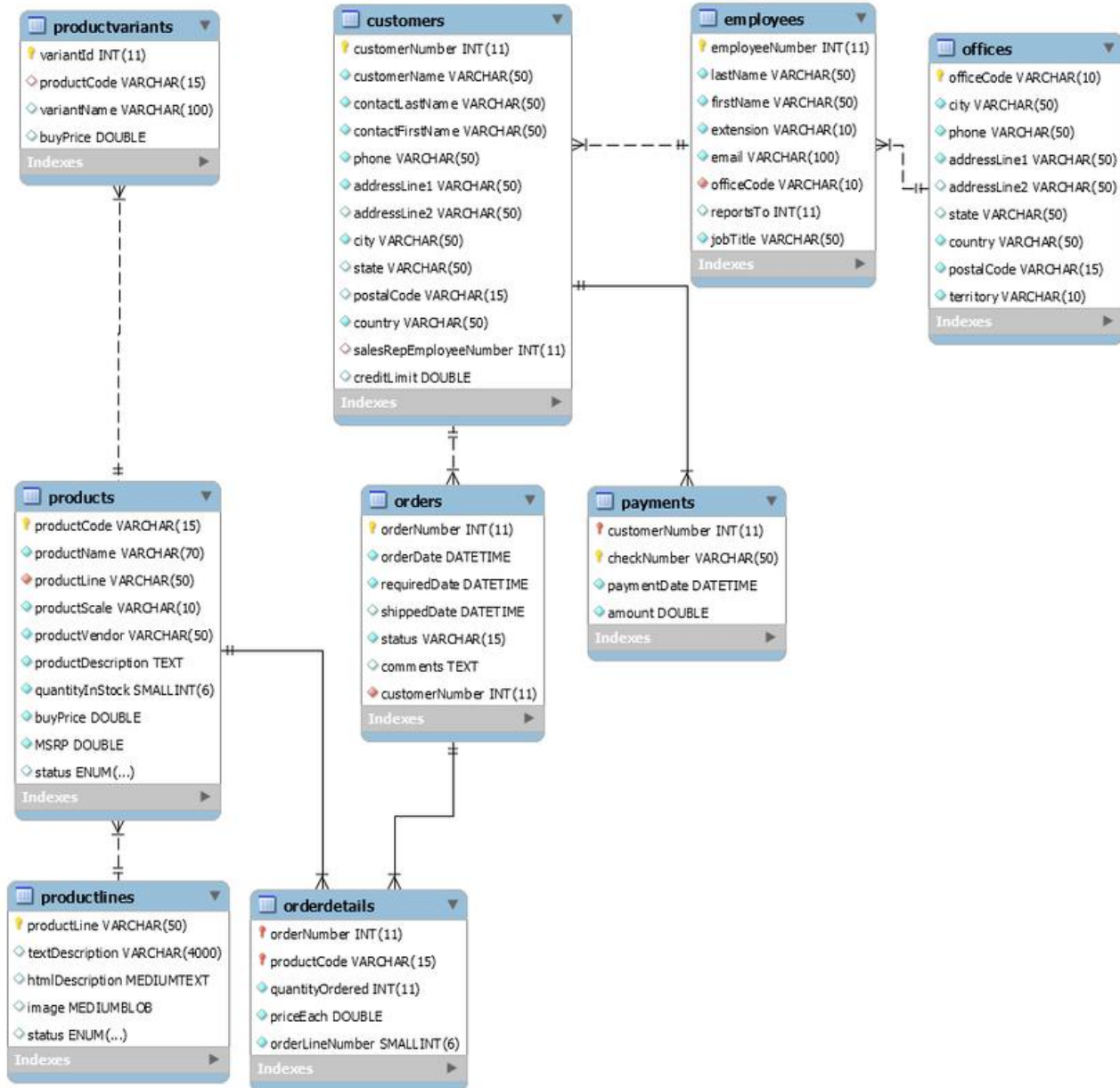
- Tránh sử dụng các cột đã đánh index với function
 - Ví dụ: `select count(*) from orders where YEAR(finished_at) = '2018'`
 - Sử dụng function YEAR cùng với cột finished_at nó sẽ không cho phép database sử dụng index ở cột finished_at bởi vì index giá trị của finished_at chứ không phải YEAR(finished_at).
 - Để có thể tránh được điều này có thể sử dụng index cho function bằng generated columns
 - Hoặc một cách khác là tìm cách để viết lại câu truy vấn tương đương mà không phải sử dụng đến function
- Chỉ select các cột thật sự cần thiết
 - Thay vì sử dụng `SELECT *` thì chỉ nên select các cột mà cần sử dụng, điều này sẽ giúp hiệu suất câu truy vấn tăng lên trong quá trình làm việc.

Chiến lược tối ưu truy vấn

- Dùng inner join thay vì outer join khi có thể
 - Sử dụng outer join quá nhiều sẽ làm hiệu suất câu truy vấn giảm đi đáng kể thay vì dùng với inner join, trong các trường hợp tương đương nhau hãy sử dụng inner join thay vì outer join
- Dùng DISTINCT và UNION chỉ khi cần
 - Khi sử dụng union và distinct trong trường hợp không cần thiết có thể dẫn đến giảm hiệu năng của câu truy vấn. Thay vì sử dụng UNION có thể sử dụng UNION ALL sẽ cho kết quả tốt hơn.
- Tránh sử dụng điều kiện với loại khác kiểu
 - Ví dụ khi so sánh bằng với where một bên là kiểu varchar và kiểu số thì index sẽ không có tác dụng do sẽ phải cast ngầm kiểu để đồng nhất dữ liệu. Trong trường hợp này, cố gắng để kiểu so sánh giống nhau ngay từ đầu sẽ cho kết quả tốt hơn.
- Sử dụng view và Stored Procedure thay cho các câu lệnh truy vấn phức tạp
 - Thời gian thực thi không khác nhau nhiều tuy nhiên việc dùng view và stored procedure sẽ giúp cho request từ client được gửi tới server nhanh hơn vì chỉ phải gọi view/stored procedure thay vì query dài

Thực hành tối ưu truy vấn

- Cơ sở dữ liệu minh họa:



Nội dung

1. Giới thiệu về database tuning
2. Tối ưu hóa truy vấn dữ liệu (query tuning)
-  3. Sử dụng INDEX
4. Giới thiệu Stored procedure và Trigger

Giới thiệu về index

- Sử dụng chỉ mục để tăng tốc độ truy vấn đến dữ liệu trong bảng.
 - Việc truy cập dữ liệu có sử dụng index gọi là 'index access'
 - Trường hợp ngược lại, gọi là 'table scan' khi đó các bản ghi sẽ được xử lý tuần tự. Table scan là một trong những động tác có hại nhất cho hiệu suất của truy vấn.
- Một chỉ mục là một cấu trúc dữ liệu giúp tổ chức các bản ghi dữ liệu trên đĩa để tối ưu các phép toán truy cập. Chúng cung cấp một phương pháp giúp nhanh chóng tìm kiếm dữ liệu dựa trên các giá trị trong các cột.
- Các lệnh INSERT và UPDATE tốn nhiều thời gian hơn trên các bảng có chỉ mục trong khi các lệnh SELECT trở nên nhanh hơn trên các bảng này. Lý do là vì, trong khi chèn và cập nhật, cơ sở dữ liệu cũng phải cần chèn hoặc cập nhật các giá trị chỉ mục.
- Ví dụ: Nếu tạo một Index trên cột khóa chính và sau đó tìm kiếm một dòng dữ liệu dựa trên một trong các giá trị của cột này:
 - Đầu tiên SQL Server sẽ tìm giá trị này trong Index.
 - Sau đó sử dụng Index để nhanh chóng xác định vị trí của dòng dữ liệu cần tìm.

Giới thiệu về index

- Các loại Index:
 - Clustered index
 - Non-Clustered index
 - Unique index
 - Composite index
- Các SQL database thường sẽ tổ chức index dưới 2 dạng:
 - B-tree: Dựa theo kiến trúc của cây cân bằng. Hỗ trợ đa dạng query.
 - Hash: Dựa theo cấu trúc dữ liệu Hash-Table. Dạng này sẽ hỗ trợ truy vấn dạng = rất tốt nhưng hỗ trợ kém hơn với truy vấn dạng range query (>,<,>=<=)

Các lệnh cơ bản với index

- Tạo chỉ mục (Câu lệnh CREATE INDEX)
 - Cú pháp: CREATE [UNIQUE] INDEX <tên chỉ mục> ON <bảng> (<thuộc tính>), ... [WITH {PRIMARY | DISALLOW NULL | IGNORE NULL }]
- Trong đó:
 - UNIQUE: các giá trị của thuộc tính làm chỉ mục không được trùng lặp nhau.
 - PRIMARY: tạo chỉ mục là khóa chính, khi chọn PRIMARY có thể bỏ qua UNIQUE.
 - DISALLOW NULL: thuộc tính làm chỉ mục không được nhận giá trị rỗng (null).
 - IGNORE NULL: thuộc tính làm chỉ mục có thể nhận giá trị rỗng (null).
- Hiển thị thông tin chỉ mục: SHOW INDEX FROM ten_bang

Các lệnh cơ bản với index

■ Xóa chỉ mục (Câu lệnh DROP INDEX)

- Cú pháp: DROP INDEX <tên chỉ mục> ON <bảng>
- Ví dụ:
 - Xóa chỉ mục (khóa) MaHocvien trên bảng Hocvien:
 - DROP INDEX MaHocvien ON Hocvien;

■ Lệnh ALTER để thêm và xóa INDEX

- ALTER TABLE ten_bang ADD PRIMARY KEY (danh_sach_cot): Lệnh này thêm một PRIMARY KEY, nghĩa là các giá trị được lập chỉ mục phải là duy nhất và không thể là NULL.
- ALTER TABLE ten_bang ADD UNIQUE ten_chi_muc (danh_sach_cot): Lệnh này tạo một chỉ mục cho các giá trị để giá trị đó phải là duy nhất (với giá trị NULL là ngoại lệ, chúng có thể xuất hiện nhiều lần).
- ALTER TABLE ten_bang ADD INDEX ten_chi_muc (danh_sach_cot): Lệnh này thêm một chỉ mục thông thường, trong đó bất kỳ giá trị nào có thể xuất hiện nhiều hơn một lần.
- ALTER TABLE ten_bang ADD FULLTEXT ten_chi_muc (danh_sach_cot): Lệnh này tạo một chỉ mục FULLTEXT đặc biệt, được sử dụng cho mục đích tìm kiếm văn bản.

Sử dụng index

- Bảng (table) phải có khóa chính (Primary Key);
- Bảng (table) phải có ít nhất 01 clustered index;
- Bảng (table) phải có số lượng non-clustered index phù hợp;
- Non-clustered index phải được tạo trên các cột (column) của bảng (table) dựa vào nhu cầu truy vấn.
- Dựa theo sự sắp xếp thứ tự như sau khi có bất kỳ index được tạo: a) WHERE clause, b) JOIN clause, c) ORDER BY clause, d) SELECT clause.
- Cột dùng để đánh index thì nên ít giá trị Null vì giá trị này gây khó cho database trong quá trình tạo cây index

Sử dụng index **đúng đắn** giúp tăng hiệu năng truy vấn

Sử dụng index

- Ví dụ 1: bảng Nhân viên chỉ có index trên khóa chính MaNhanVien và khóa ngoại MaPhong.
 - Truy vấn tìm kiếm với tên nhân viên được thực hiện thường xuyên: `SELECT * FROM NhanVien WHERE TenNhanVien='Hung';`
 - Vấn đề: Truy vấn không có index trên cột TenNhanVien
- Ví dụ 2: bảng Nhân viên có non-clustering index trên cột Luong
 - Truy vấn tính toán với cột Lương thực hiện chậm: `SELECT * FROM NhanVien WHERE Luong/12=400;`
 - Vấn đề: index không được sử dụng vì cột được đặt trong biểu thức số học.

Thực hành sử dụng index

- Xem xét sử dụng index trong các truy vấn và tối ưu phù hợp:

1. `SELECT HoNhanVien, TenNhanVien FROM NhanVien WHERE MaNhanVien = '123456789';`

2. `SELECT HoNhanVien, TenNhanVien FROM NhanVien WHERE MaNhanVien = '123456789' and MaPhong = 5;`

3. `SELECT HoNhanVien, TenNhanVien FROM NhanVien WHERE Luong between 123 and 500 and MaPhong = 5;`

4. `SELECT HoNhanVien, TenNhanVien, NgaySinh FROM NhanVien WHERE UPPER(HoNhanVien) = "NGUYEN";`

5. `SELECT HoNhanVien, TenNhanVien, NgaySinh FROM NhanVien WHERE UPPER(HoNhanVien) < "NGUYEN" and Luong > 500;`

Nội dung

- 1. Giới thiệu về database tuning**
- 2. Tối ưu hóa truy vấn dữ liệu (query tuning)**
- 3. Sử dụng INDEX**
-  **4. Giới thiệu Stored procedure và Trigger**

Stored procedure

- **Store Procedure** cho phép tập hợp nhiều câu lệnh SQL trong một chương trình con. Giống như các ngôn ngữ khác, Store Procedure cũng cho phép truyền tham số.

- Khai báo:

```
CREATE PROCEDURE sp_name ([parameter[,...]])  
  routine_body
```

```
CREATE FUNCTION sp_name ([parameter[,...]])  
  RETURNS type  
  routine_body
```

parameter:

[IN | OUT | INOUT] *param_name* *type*

type: Any valid MySQL data type

routine_body:

Valid SQL procedure statements or statements

Stored procedure

■ Ví dụ:

```
DELIMITER $$  
CREATE PROCEDURE countNV()  
BEGIN  
    SELECT      count(*) FROM NhanVien;  
END$$
```

```
CREATE PROCEDURE showNV(mp int)  
BEGIN  
    SELECT      *  
    FROM        NhanVien n  
    WHERE       n.MaPhong = mp;  
END$$  
DELIMITER ;
```

■ Sử dụng: CALL sp_name([parameter[,...]])

■ Ví dụ

- CALL countNV();
- CALL showNV(5);

Store Procedure

- **Khai báo và sử dụng biến:**
- Khai báo biến
- Tên biến phân biệt chữ hoa chữ thường. Biến không cần khởi tạo (biến chưa khởi tạo có giá trị NULL)
 - Biến do người dùng định nghĩa @var_name
 - DECLARE var_name[,...] type [DEFAULT value]
- Gán giá trị
 - Toán tử gán :=
 - SET @var_name = expr [, var_name = expr]
 - SELECT INTO
 - SELECT col_name[,...] INTO var_name[,...] table_expr
- Ví dụ:
 - **DECLARE c INT;**
 - **SELECT COUNT(*) INTO c FROM sv;**

Store Procedure

■ Biến hệ thống:

- Thiết đặt các cấu hình cho MySQL server
- 2 loại:
 - Biến toàn cục: thông tin tổng thể về hoạt động của hệ thống
 - Biến session: lưu thông tin trong phiên kết nối của người dùng
- Mỗi biến hệ thống có một giá trị mặc định, có thể thiết lập giá trị ngay trong lúc chạy server
- Một số lệnh:

```
SHOW GLOBAL VARIABLES;
```

```
SHOW SESSION VARIABLES;
```

```
SHOW GLOBAL VARIABLES LIKE 'SOMENAME%';
```

```
SELECT @@GLOBAL.var_name;
```

```
SELECT @@session_var_name;
```

Store Procedure

- **Các cấu trúc điều khiển:**
- Các cấu trúc được hỗ trợ:
 - Cấu trúc rẽ nhánh: IF, CASE,
 - Cấu trúc lặp: LOOP, WHILE, ITERATE, LEAVE
- **Cấu trúc rẽ nhánh:**

```
IF search_condition THEN statement_list
  [ELSEIF search_condition THEN statement_list] ...
  [ELSE statement_list]
END IF
```

```
CASE case_value
  WHEN when_value THEN statement_list
  [WHEN when_value THEN statement_list ...]
  [ELSE statement_list]
END CASE
```

Store Procedure

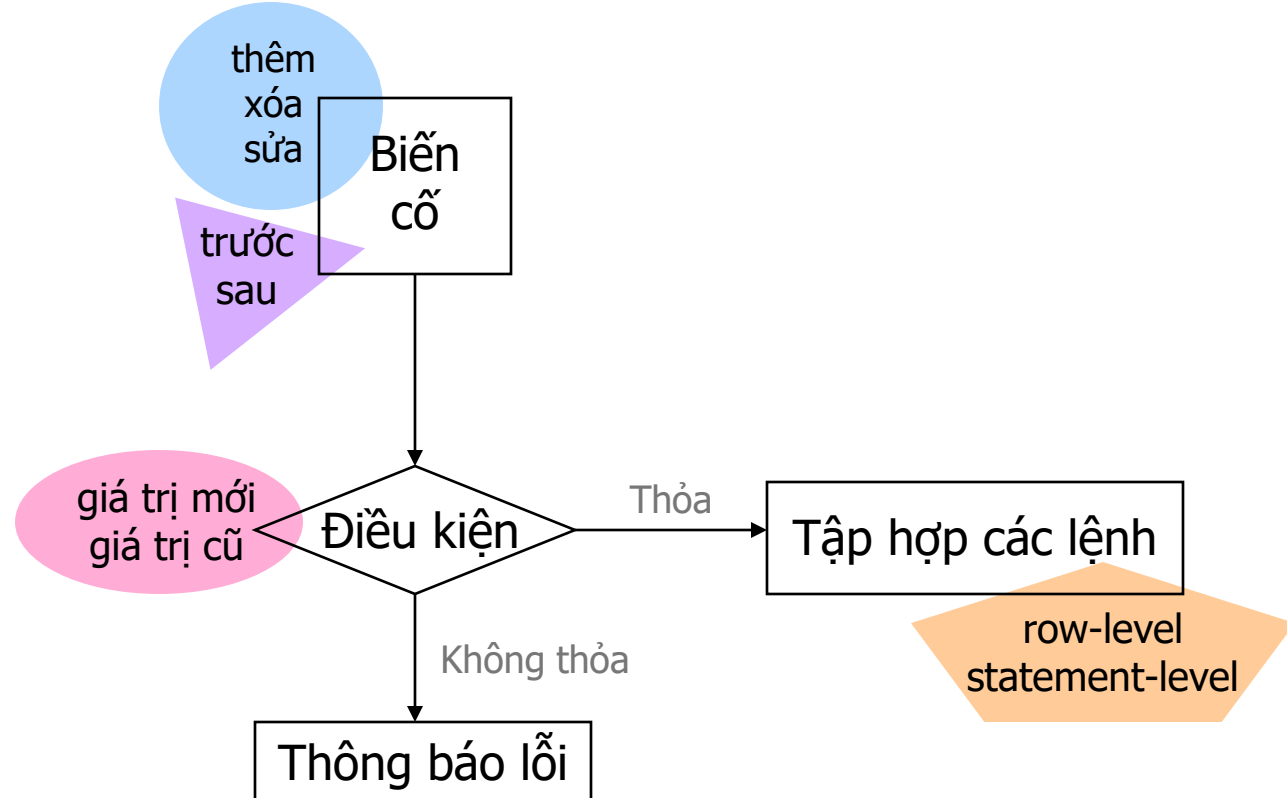
- Các cấu trúc điều khiển:
- Cấu trúc lặp:

```
[begin_label:] REPEAT  
    statement_list  
UNTIL search_condition  
END REPEAT [end_label]
```

```
[begin_label:] WHILE search_condition DO  
    statement_list  
END WHILE [end_label]
```

Trigger

- **Trigger** là tập hợp các lệnh được thực hiện tự động khi xuất hiện một biến cố nào đó



Trigger

- **Trigger** là một đối tượng liên kết với bảng (table), được kích hoạt khi có những sự kiện (thêm, xóa, sửa...) xảy ra trong bảng.
- Tạo Trigger:

```
CREATE TRIGGER trigger_name  
    [BEFORE | AFTER] trigger_event  
ON tbl_name FOR EACH ROW  
    trigger_stmt
```

- trigger_event: Xác định loại sự kiện (INSERT, UPDATE, DELETE)
- trigger_stmt: Các lệnh được gọi khi trigger bị kích hoạt, để sử dụng nhiều lệnh, dùng cấu trúc BEGIN...END giống như STORE PROCEDURE

Trigger

- **Sử dụng Trigger:**

- Khi các Trigger được kích hoạt, để lấy các giá trị của bảng trước và sau khi sự kiện xảy ra, MySQL sử dụng các từ khóa OLD, NEW
 - OLD: đại diện cho dòng bị DELETE hay dòng trước khi UPDATE
 - NEW: đại diện cho dòng được INSERT vào hay dòng sau khi UPDATE
 - Dữ liệu trong dòng OLD là readonly
 - Dữ liệu trong dòng NEW có thể chỉnh sửa (sử dụng để kiểm tra dữ liệu trong các câu lệnh INSERT và UPDATE với sự kiện loại BEFORE)

Thực hành Stored Procedure và Trigger

- Xây dựng một thủ tục nhận tham số vào là một giá trị mã phòng ban, kiểm tra xem mã phòng truyền vào này có tồn tại không? Nếu có thì trả về một tham số ra chưa giá trị là số nhân viên của phòng ban có mã tương ứng, đồng thời liệt kê danh sách nhân viên của phòng ban này. Nếu không tồn tại thì hiển thị một thông báo.
- Xây dựng trigger trên bảng nhân viên. Mục đích của trigger là ghi lại nhật ký sự kiện cập nhật trên bảng nhân viên vào một bảng log, trong đó cần ghi lại mã nhân viên bị cập nhật, giá trị lương (lương cũ - trước khi update và lương mới - sau khi update), thời điểm update, tài khoản thực hiện update.

